

可程式邏輯設計期末專題 | 數字密碼鎖實作提示詞

專題概述

請幫我實作一個「數字密碼鎖」的數位邏輯電路，使用 **Verilog HDL** 撰寫，並以 **ModelSim**（或 **Vivado 模擬器**）進行功能模擬，不需要燒錄至 **FPGA** 實體板。

系統功能規格

功能描述

使用者透過按鍵依序輸入 4 位數字密碼（每位 0-9），系統比對是否與預設密碼吻合：

- 正確**：顯示解鎖狀態，`led_unlock` 拉高
- 錯誤**：顯示錯誤提示，累計錯誤次數
- 連續錯誤 3 次**：進入鎖定狀態，停止接受輸入，直到按下 `rst`

輸入／輸出規格（精簡設計原則）

原則：信號越少越好，每個輸入都必須有其必要性。

輸入 (Inputs)

信號名稱	位元寬	說明
<code>clk</code>	1-bit	系統時脈
<code>rst</code>	1-bit	非同步重置（高電位有效），清除所有狀態
<code>key_in</code>	4-bit	使用者輸入的數字（ <code>0000</code> – <code>1001</code> 代表 0-9，其餘忽略）
<code>key_valid</code>	1-bit	高電位脈衝，代表目前 <code>key_in</code> 是一筆有效輸入（模擬按鍵按下）

說明：不使用實體矩陣鍵盤掃描電路，改以 `key_in + key_valid` 組合模擬按鍵輸入，在 `testbench` 中直接驅動，符合純模擬需求。

輸出 (Outputs)

信號名稱	位元寬	說明
<code>seg_digit</code>	4-bit	目前已輸入幾位（顯示 0-4，直接以數值輸出）

信號名稱	位元寬	說明
led_unlock	1-bit	解鎖成功指示燈（高電位亮起）
led_error	1-bit	本次輸入錯誤指示燈（一個 clock 週期後自動清除）
led_locked	1-bit	系統鎖定中（連續 3 次錯誤後持續高電位）
error_count	2-bit	當前錯誤次數（0-3，供 testbench 驗證用）

預設密碼

```
verilog

// 在模組頂端以 parameter 定義，方便修改
parameter DIGIT0 = 4'd1;
parameter DIGIT1 = 4'd2;
parameter DIGIT2 = 4'd3;
parameter DIGIT3 = 4'd4;
// 預設密碼為 1-2-3-4
```

有限狀態機（FSM）設計

請以 FSM 架構實作主控邏輯，共 8 個狀態：

狀態	說明
IDLE	等待第一位輸入，seg_digit = 0
INPUT_1	已收到第 1 位，等待第 2 位，seg_digit = 1
INPUT_2	已收到第 2 位，等待第 3 位，seg_digit = 2
INPUT_3	已收到第 3 位，等待第 4 位，seg_digit = 3
CHECK	收到第 4 位後，比對全部 4 位密碼（一個 clk 週期）
UNLOCKED	密碼正確，led_unlock = 1，等待 rst 重置
ERROR	密碼錯誤，led_error = 1 一拍，error_count++，視錯誤次數轉至 IDLE 或 LOCKED
LOCKED	連續錯誤 3 次，led_locked = 1，僅接受 rst

- 僅當 `key_valid = 1` 時，才從 `IDLE` → `INPUT_1` → `INPUT_2` → `INPUT_3` → `CHECK` 依序推進
 - `rst` 可從任意狀態回到 `IDLE`，同時清除 `error_count` 與所有輸出
 - `CHECK` 狀態：若 4 位全符合 → `UNLOCKED`；否則 → `ERROR`
 - `ERROR` 狀態：若 `error_count` 達 3 → `LOCKED`；否則 → `IDLE`
-

模組介面 (Module Port)

```
verilog
module digital_lock (
    input wire      clk,
    input wire      rst,
    input wire [3:0] key_in,
    input wire      key_valid,
    output reg [3:0] seg_digit,
    output reg      led_unlock,
    output reg      led_error,
    output reg      led_locked,
    output reg [1:0] error_count
);
```

Testbench 撰寫需求

請一併提供 `tb_digital_lock.v`，需涵蓋以下測試情境：

1. **正確密碼測試**：依序輸入 1→2→3→4，驗證 `led_unlock` 拉高
2. **單次錯誤測試**：輸入 1→2→3→5（最後一位錯），驗證 `led_error` 拉高一拍、`error_count = 1`、系統回到 `IDLE`
3. **三次錯誤鎖定測試**：連續輸入 3 組錯誤密碼，驗證 `led_locked` 持續高電位、後續按鍵無效
4. **鎖定後重置測試**：在 `LOCKED` 狀態拉高 `rst`，驗證所有輸出歸零、可重新輸入
5. **中途重置測試**：輸入一半時拉高 `rst`，驗證 `seg_digit` 歸零

Testbench 時序建議

```
verilog
```

```
// 時脈產生
always #5 clk = ~clk; // 10ns 週期，100MHz

// 模擬按鍵輸入的 task
task press_key;
    input [3:0] digit;
    begin
        @(posedge clk);
        key_in    = digit;
        key_valid = 1'b1;
        @(posedge clk);
        key_valid = 1'b0;
        #20; // 模擬去彈跳間隔
    end
endtask
```

程式碼規範

- 使用 **Verilog-2001** 語法（避免 SystemVerilog 特性，確保 ModelSim 相容）
- FSM 採用**三段式**寫法（現態暫存器、次態邏輯、輸出邏輯分開）
- 所有暫存器在 `(rst)` 時需明確初始化
- 加入適量中文或英文註解說明狀態轉移邏輯
- 模組名稱：`digital_lock`，testbench 名稱：`tb_digital_lock`

繳交檔案清單

請輸出以下兩個檔案的**完整可執行程式碼**，不得省略任何部分：

1. `digital_lock.v` — 主模組
2. `tb_digital_lock.v` — 測試平台，含上述 5 項測試情境與 `$display` 驗證輸出

補充說明

- 本專題為純模擬，**不需要** 7-segment 解碼器、矩陣鍵盤掃描、去彈跳電路等實體硬體邏輯
- `seg_digit` 直接以 4-bit 二進位數值輸出即可，testbench 以 `$display` 確認
- 密碼以 `parameter` 定義，組員可自行修改為其他四位數字